



MINISTÈRE CHARGÉ
DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ *Cosson*
Nom d'usage ▶
Prénom ▶ *Anthony*
Adresse ▶ *10 rue titre réussi 14540 Soliers City*

Titre professionnel visé

Développeur Web et Web Mobile

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Activité-type 1 : Développer la partie front-end d'une application web ou web mobile sécurisée.	p.5
➤ CP 1 Installer et configurer son environnement de travail en fonction du projet web ou web mobile	p.5
➤ CP 2 Maquetter des interfaces utilisateur web ou web mobile	p.7
➤ CP 3 Réaliser des interfaces utilisateur statiques web ou web mobile	p.11
➤ CP 4 Développer la partie dynamique des interfaces utilisateur web ou web mobile	p.15
Intitulé de l'activité-type n° 2 : Développer la partie back-end d'une application web ou web mobile sécurisée.	p.19
➤ CP 5 Mettre en place une base de données relationnelle	p.19
➤ CP 6 Développer des composants d'accès aux données SQL et NoSQL	p.22
➤ CP 7 Développer des composants métier côté serveur	p.25
➤ CP 8 Documenter le déploiement d'une application dynamique web ou web mobile	p.32
Titres, diplômes, CQP, attestations de formation (facultatif)	p.36
Déclaration sur l'honneur	p.37
Documents illustrant la pratique professionnelle (facultatif)	NA
Annexes (Si le RC le prévoit)	p.38

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée.

CP 1 - Installer et configurer son environnement de travail en fonction du projet web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Cette tâche a été faite dans un cadre général, au tout début de la formation sur une machine virtuelle puis pendant la préparation du dossier, directement sur mon ordinateur pour installer les bases de mon environnement de code.

1. Installer l'éditeur de code VSC selon le site officiel : <https://code.visualstudio.com/download>
2. Installer Git selon le site officiel : <https://git-scm.com/downloads>
3. Installer des extensions VSC (en allant voir celles qui étaient installées par l'école sur la VM et sélectionnant celles que je voulais retrouver sur mon ordinateur) :
 - a. *Carbon et CodeSnap* (pour les captures d'écran des dossiers du TP) ;
 - b. *Auto Rename Tag, Babel JavaScript, Better Comments, ESLint, Markdown Preview Github Styling, markdownlint, php cs fixer, PHP Getters & Setters, PHP Intelephense, PHP Namespace Resolver, Prettier - Code formatter, TODO Highlight* (pour l'aide syntaxique et l'écriture d'un code propre) ;
 - c. *Duplicate action, Material Icon Theme, ThunderClient* (pour l'amélioration de l'outil VSC).
4. Configurer Git pour l'authentification auprès de Github via le protocole SSH en suivant la documentation Github <https://docs.github.com/fr/authentication/> :

J'ai généré une nouvelle clé SSH pour mon ordinateur depuis le terminal de VSC (mais ça aurait pu être fait depuis un terminal n'importe où) :

```
ssh-keygen -t ed25519 -C "antxxx.cosxxxxx@gmail.com"
```

Je l'ai enregistrée dans le dossier proposé par défaut et j'ai renseigné une phrase secrète comme demandé.

Dans un nouveau terminal avec les privilèges administrateur (clic droit "Exécuter en tant qu'administrateur" sur l'icône de programme Git Bash) j'ai exécuté les commandes pour démarrer manuellement l'agent ssh :

```
Get-Service -Name ssh-agent | Set-Service -StartupType Manual
Start-Service ssh-agent
```

De retour dans le terminal "normal" (ici celui de VSC), j'ai ajouté la clé privée SSH à l'agent SSH :

```
ssh-add c:/Users/antxxx/.ssh/id_ed25519
```

J'ai copié la clé publique dans le presse papier pour l'ajouter facilement dans les paramètres de mon compte Github avec :

```
clip < ~/.ssh/id_ed25519.pub
```

Le fait de sécuriser avec SSH me permet surtout de ne pas avoir à rentrer mon nom d'utilisateur et personal access token (mais quand même une phrase secrète comme j'en ai mis une). C'est un niveau de sécurité probablement trop élevé par rapport à la sensibilité des informations sur mon compte Github.

5. Installation d'une distribution d'Apache (XAMPP) pour développer et tester sur un serveur web (représente mieux un vrai environnement que l'utilisation du fichier local directement ouvert dans le navigateur). Configuration du DocumentRoot vers un dossier "Code" de mon ordinateur dans le fichier *httpd.conf*. grâce au XAMPP Control Panel.

2. Précisez les moyens utilisés :

J'ai utilisé :

- les site des logiciels <https://code.visualstudio.com/download> et <https://git-scm.com/download> ;
- la documentation Github : <https://docs.github.com/fr/get-started/getting-started-with-git/>;
- ainsi que le terminal dans VSC et directement dans l'application Gitbash.

3. Avec qui avez-vous travaillé ?

Pour cette installation sur mon ordinateur, j'ai travaillé seule.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *École O'Clock*

Chantier, atelier, service ▶ *Au global de la formation*

Période d'exercice ▶ Du : *19/01/2024* au : *19/07/2024 + 28-29/07/2024*

5. Informations complémentaires (facultatif)

- J'ai également installé Docker pour le futur et la CP8. Docker est un gestionnaire de conteneurs permettant d'avoir l'environnement nécessaire au code sans avoir à réinstaller ou mettre à jour les logiciels nécessaires sur chaque ordinateur pour faire fonctionner le site.
- Pour le travail en équipe, nous avons également installé Live Share mais je ne l'ai pas encore mis sur le VSC de mon ordinateur.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée.

CP 2 - Maquetter des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre d'un exercice de ma formation, j'ai réalisé des wireframes et user-stories à partir d'une liste de fonctionnalités données pour la partie front du MVP d'un nouveau projet de site web de recette de cuisine en responsive. C'est une des premières étapes d'un projet qui permet de visualiser ce que sera le site, de s'assurer que le brief est bien compris avec le client, de déceler de potentiels premières incohérences dans l'expérience utilisateur et de créer une base saine pour commencer à coder de façon efficace.

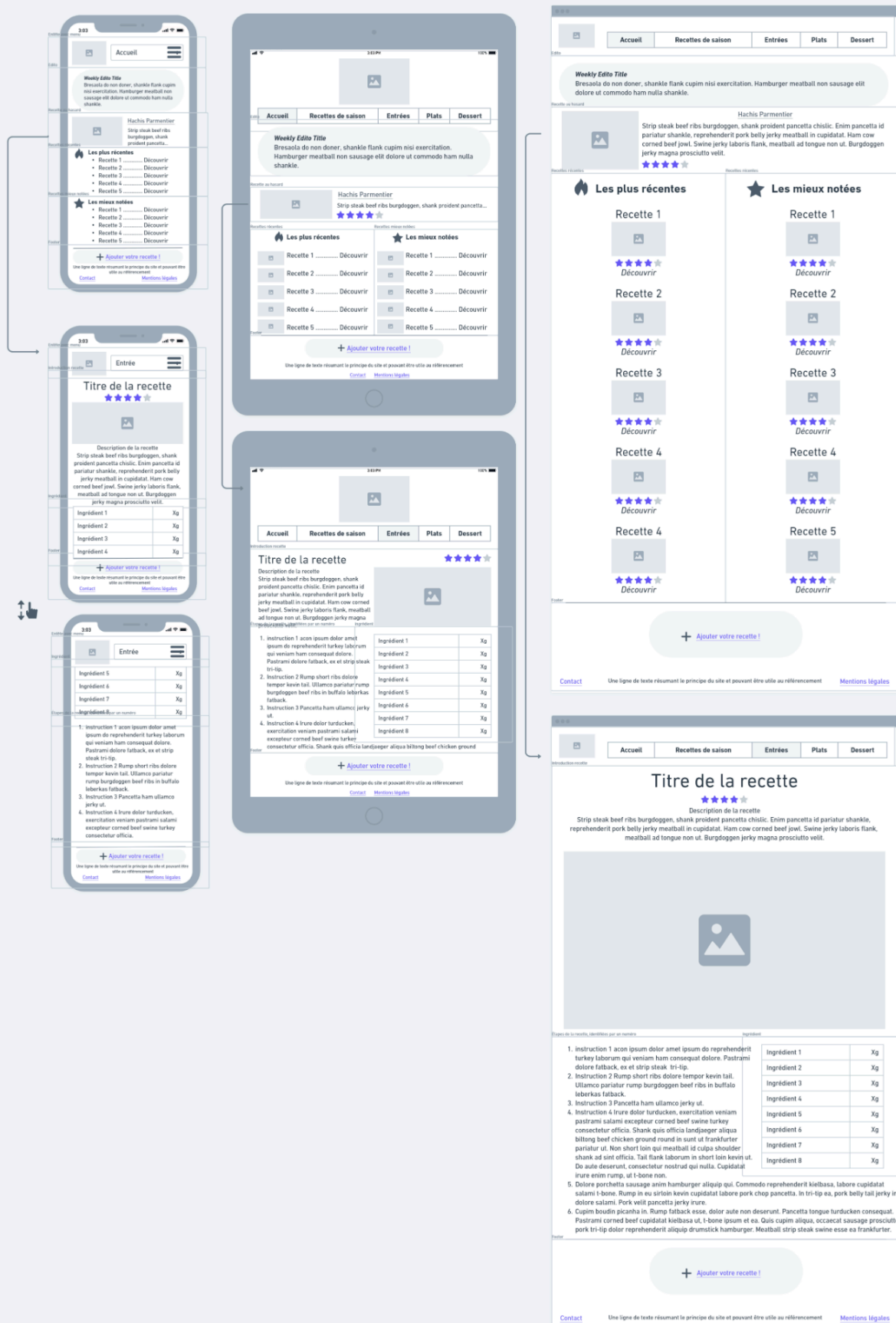
Wireframes :

- 2 pages : accueil et page recette
- 3 formats : mobile, tablette et desktop
- Ils ne représentent que la disposition schématique des éléments et interactions d'une page web, ils ne prennent pas encore en compte la charte graphique car ils sont uniquement fonctionnels.

Sur ces wireframes :

- les flèches indiquent la navigation entre les pages (hors navigation principale) ; les textes ont été mis en *lorem ipsum* plutôt qu'en barres remplies ;
- les indications de ce que sont les blocs sont en commentaires sur l'application mais il est plus optimal de les mettre en tant qu'éléments visibles à 1ère vue, comme je l'ai fait sur les mêmes wireframes réalisés avec d'autres outils (annexes 2.1 et 2.2).

J'ai commencé par les wireframes mobile dans une logique mobile-first car il est plus efficace de travailler sur un petit espace puis d'agrandir, remplir ou rajouter des choses si besoin sur un plus grand espace plutôt que l'inverse.



User Stories :

J'ai ensuite traduit les fonctionnalités du cahier des charges en user stories. Les User stories sont des phrases courtes décrivant des scénarios d'utilisation du site, de façon très unitaire. Elles décrivent en détail chaque fonctionnalité en exprimant clairement quel rôle va utiliser quelle fonctionnalité et dans quel but. Ces users stories sont une façon efficace de découper les tâches pour les développeurs. En effet, cette technique met l'utilisateur au centre du produit et permet de délivrer des fonctionnalités finies rapidement.

En tant que	Je veux	Afin de (si besoin/nécessaire)
visiteur	Je veux pouvoir accéder à la liste des ingrédients d'une recette	afin de réaliser celle-ci dans ma cuisine
visiteur	Je veux pouvoir accéder aux étapes numérotées d'une recette	afin de réaliser celle-ci dans ma cuisine
visiteur	Je veux pouvoir accéder aux recettes selon des catégories	afin de trouver facilement une recette adaptée à mon besoin
visiteur	Je veux découvrir de nouvelles recettes	afin de diversifier mes menus
visiteur	Je veux voir les dernières recettes ajoutées	afin d'être au courant des nouveautés
visiteur	Je veux voir les recettes les mieux notées	afin de trouver des recettes approuvées
visiteur	Je veux pouvoir proposer une recette	afin de partager mes connaissances
visiteur	Je veux pouvoir noter une recette	afin d'aider les autres visiteurs
admin	Je veux pouvoir ajouter de nouvelles recettes	
admin	Je veux pouvoir supprimer des recettes	
admin	Je veux pouvoir modifier des recettes	
admin	Je veux pouvoir classer les recettes dans des catégories	
admin	Je veux pouvoir valider les recettes proposés par les visiteurs	
admin	Je veux pouvoir modifier l'edito	
admin	Je veux pouvoir modifier la ligne de texte résumant le principe du site	

2. Précisez les moyens utilisés :

Pour les wireframes, j'ai profité de cet exercice pour essayer différents outils : draw.io, whimsical, figma, wireframes.cc et mockflow. Finalement, j'ai préféré <https://whimsical.com/> car il restait très simple et parce qu'il limite l'envie de rajouter du design (contrairement à Mockflow par exemple).

Pour les users stories, je me suis servie d'un fichier Markdown directement dans le projet, avec la syntaxe des tableaux :



```

- □ ×
| Titre | Titre |
| -- | -- |
| Texte | Texte |

```

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule, dans le cadre d'un exercice de formation le soir après les cours, non corrigé.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *École O'Clock*

Chantier, atelier, service ▶ *Exercice de formation "challenge-documents-projet"*

Période d'exercice ▶ Du : *13/05/2024* au : *14/05/2024*

5. Informations complémentaires (facultatif)

Voici le "cahier des charges" minimaliste qui était donné en consigne d'exercice :

10 Minutes O'Four - Documents d'un projet

Travaillons sur la préparation d'un nouveau projet, un site en Responsive Web Design qui regroupera des recettes de cuisine. Ici, il s'agit d'un MVP (minimum viable product), et donc les fonctionnalités à implémenter dans un premier temps sont assez peu nombreuses.

Voici la liste des fonctionnalités attendues sur cette première version :

- En haut de la page :
 - il faut afficher un logo (au clic sur ce logo, on revient sur la page d'accueil)
 - le menu doit permettre de naviguer sur les pages principales : accueil, recettes de saison, entrées, plats, desserts
- Sur la page d'accueil :
 - on doit retrouver un edito (un petit paragraphe susceptible de changer de façon hebdomadaire)
 - on doit trouver un lien vers une recette au hasard. A chaque fois qu'on rafraîchit la page, on affiche une recette différente
 - il faut afficher une liste des 5 recettes les plus récentes
 - il faut afficher une liste des 5 recettes les mieux notées
- Dans le pied de page :
 - on veut afficher des liens vers la page de contact, les mentions légales et vers le formulaire permettant de proposer une recette, ainsi qu'une ligne de texte résumant le principe du site et pouvant être utile au référencement (SEO)
- Sur la page de recette :
 - on doit voir la liste des ingrédients séparée du reste de la recette
 - on doit voir les étapes de la recette identifiée notamment par un numéro

Étant donné qu'on n'a pas encore de back-office, il n'y a pas de rôle particulier pour chaque fonctionnalité, cela sera vu en V2.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée.

CP 3 - Réaliser des interfaces utilisateur statiques web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Cette tâche consistait à réaliser les bases d'une intégration responsive de la page d'accueil d'un blog pour un client fictif, selon un cahier des charges et des wireframes donnés (en annexe 3.1 et 3.2).

Un point important de la demande du client était d'avoir un site web responsive en mobile-first. C'est-à-dire un site dont le front est d'abord codé pour une taille d'écran de téléphone puis décliné sur tablette et ordinateur. Travailler en mobile-first permet de s'assurer que la plus petite version de l'intégration sera lisible et pertinente tout en ayant une cohérence entre les différentes tailles, sans avoir à faire rentrer des éléments au chausse pied.

Pour ce projet, nous devions coder cette première tâche de mise en place de l'intégration statique grâce à Bootstrap qui permet de faire facilement un site responsive en mobile-first.

```

14 > <header>...
42 </header>
43
44 <main>
45 <div class="container-fluid text-center m-md-0">
46 <div class="row border">
47 <div class="my-3">A la dérive</div>
48 </div>
49 <div class="row">...
51 </div>
52 </div>
53 <div class="d-lg-flex">
54 <section class="col-lg-9">
55 <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">
56 <h2 class="card-header">Article title</h2>
57 <div class="card-body">
58 <p class="card-text">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Quar
    reiciendis, vel itaque sunt quibusdam autem nisi modi voluptatum, incidunt consequ
    <a href="#" aria-label="Meta info on article XXX" class="card-link">Meta info</a>
59 </div>
60 </article>
61 <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
62 </article>
63 <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
64 </article>
65 <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
66 </article>
67 <nav class="container mx-auto p-3 col-11 col-md-9 me-lg-0 border d-flex justify-content-be
68 </nav>
69 </section>
70 <aside class="col-lg-3">...
71 </aside>
72 </div>
73 </main>
74
75 <footer class="m-lg-5">...
76 </footer>

```

Nous avons tout d'abord réalisé la structure de la page en html en utilisant un bon niveau de balisage sémantique même si le cahier des charges spécifiait que "le but visé est surtout pédagogique (écriture et partage d'articles en interne dans l'école)".

Le code est séparé en header/main/footer.

Les liens dont le texte entre balise n'est pas explicite bénéficie d'un aria-label pour l'accessibilité.

Les éléments nécessaires à la mise en page sont des div.

Avec Bootstrap, le style de la page passe principalement par des classes spécifiques à la librairie Bootstrap, appliquées aux balises. La dimension responsive est apportée par l'intégration de -md- ou

-lg- dans ces classes pour préciser les différences de mise en page pour les tablettes (breakpoint $\geq 768\text{px}$) et les ordinateurs ($\geq 992\text{px}$).

Ci-dessous, par défaut car Bootstrap encourage le mobile-first, la class navbar est collapsed (=repliée) et c'est avec navbar-expand-md qu'on la "déplie" à partir du breakpoint des tablettes.

```
14 | <header>
15 |   <div class="container-fluid text-center m-md-0">
16 |     <div class="row">
17 | >   <nav class="navbar navbar-expand-md container-fluid border p-0">...
38 |     </nav>
39 |   </div>
40 | </div>
41 | </header>
```

Et ci-dessous on change la disposition des blocs <section> et <aside> quand on passe sur ordinateur pour que l'<aside> se place à droite grâce à une flexbox (d-lg-flex) mise sur la div englobant les deux. On change également la proportion de la page prise par la <section>, les <article> et l'<aside> selon la taille d'écran grâce aux classes col-XX(-md/lg), dont le chiffre représente une proportion des 12 colonnes des containers bootstraps (qui forment la largeur).

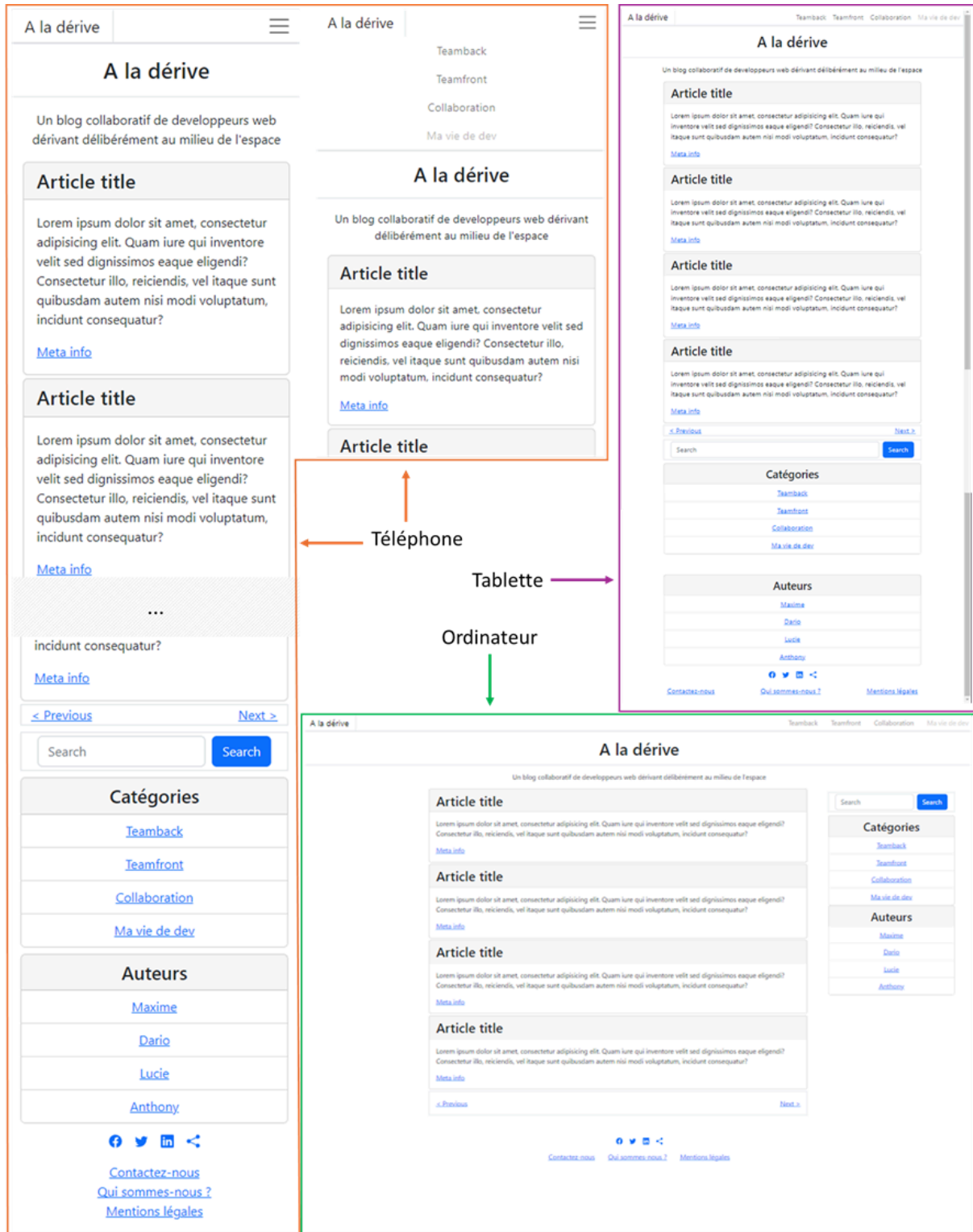
```
43 | <main>
44 | > <div class="container-fluid text-center m-md-0">...
51 | </div>
52 | <div class="d-lg-flex">
53 |   <section class="col-lg-9">
54 | >   <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
60 |   </article>
61 | >   <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
67 |   </article>
68 | >   <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
74 |   </article>
75 | >   <article class="card mx-auto col-11 mb-1 col-md-9 me-lg-0">...
81 |   </article>
82 | >   <nav class="container mx-auto p-3 col-11 col-md-9 me-lg-0 border d-flex justify-content-between">...
85 |   </nav>
86 | </section>
87 | >   <aside class="col-lg-3">...
114 |   </aside>
115 | </div>
116 | </main>
```

Des icônes sont intégrées depuis une autre librairie de Bootstrap, Bootstrap Icons. Cela permet de ne pas avoir à stocker l'image de l'icône. Bootstrap étant une source fiable, il y a peu de chance d'avoir un souci. Ces icônes sont rangées dans une <nav> pour suivre la bonne pratique de sémantique et rendre la navigation plus accessible et j'ai donné un aria-label aux liens pour l'accessibilité car le contenu est une icône.

```
120 | <nav class="d-flex justify-content-center mb-3">
121 |   <a class="px-2" aria-label="facebook" href="https://www.facebook.com">
122 | >   <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-facebook"
124 |   </svg>
125 |   </a>
126 |   <a class="px-2" aria-label="twitter x" href="https://www.x.com">
127 | >   <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-twitter"
129 |   </svg>
130 |   </a>
131 | >   <a class="px-2" aria-label="linkedin" href="https://www.linkedin.com">...
```

Voici le résultat visible, sur 3 formats d'écran, sur la page suivant :

DOSSIER PROFESSIONNEL (DP)



2. Précisez les moyens utilisés :

Cette page est réalisée en html. A cette étape du projet, nous n'avons pas mis de CSS car tout passe par des classes dans l'html grâce à Bootstrap avec Bootstrap Icons.

Pour tester la dimension responsive de la page, l'outil DevTool de Chrome a été utilisé.

Le site a été testé sur Chrome, Firefox et Edge mais pas Internet Explorer 11 comme demandé dans le cdc car ce n'est plus un navigateur qui est conseillé par Microsoft.

3. Avec qui avez-vous travaillé ?

Sur cet exercice, j'ai travaillé en pair-programming avec un autre étudiant, sur une journée, puis j'ai rajouté, seule, des petites corrections par la suite. Nous avons choisi de travailler globalement avec un *driver* (écrivant le code) et un *observer* (observant pour apporter un point de vue avec un peu plus de recul), en échangeant de rôle à mi-journée. Nous autorisons tout de même l'*observer* à écrire également, surtout pour gagner du temps sur des tâches faciles ou répétitives mais reprenons les rôles plus "strictement" pour résoudre des problèmes. Nous utilisons l'extension VSCode Live Share et avons une vision sur la page web en cours grâce au vpn proposé par les machines virtuelles de l'école installées sur nos deux ordinateurs respectifs.

J'ai par la suite re-travaillé seule le projet pour le rapprocher plus des wireframes.

4. Contexte

Nom de l'entreprise, organisme ou association - *École O'Clock*

Chantier, atelier, service - *Exercice de formation "atelier-cdc-oblog"*

Période d'exercice - Du : *06/03/2024* au : *06/03/2024*

5. Informations complémentaires (facultatif)

Cette page est une base pour un second travail d'intégration comprenant le design avec des maquettes mais cela n'était pas à faire dans le cadre de cet exercice.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée.

CP 4 - Développer la partie dynamique des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur la base d'une intégration HTML/CSS déjà réalisée, le brief était de dynamiser un site similaire à Tripadvisor en ajoutant différentes fonctionnalités :

- Bouton de changement de thème dark/normal et 3 couleurs ;
- Slider d'images ;
- Encart d'inscription à une newsletter ouvrant/fermant ;
- Tri de commentaires selon le nombre d'étoiles attribuées.

Pour rester concis, seuls la gestion des thèmes et de l'encart newsletter seront détaillés. Le code du module du Slider et de tri de commentaires se trouvent en annexe 4.1 et 4.2.

Au global :

```

v js
JS app.js
JS message.js
JS newsletter.js
JS reviews.js
JS slider.js
JS storage.js
JS theme.js

```

Les différentes fonctionnalités sont rangées dans des modules différents, afin d'avoir un code mieux organisé, plus évolutif et maintenable.

Les modules storage et message sont potentiellement utilisables pour différentes fonctionnalités.

J'ai documenté les fonctions et le code de ce projet grâce à des commentaires en anglais dans le code. Ils serviront ici de base d'explication.

Changement de thème et couleur :

```

54 /**
55  * Function that switches in between dark and normal themes
56  * - with a toggle of a class put on the body
57  * - also stores the information true (dark theme is on) or false
58  */
59 handleSwitchDarkTheme: function() {
60     theme.bodyElement.classList.toggle('theme-dark');
61     storage.setLocal(
62         "isDarkMode",
63         theme.bodyElement.classList.contains("theme-dark"))
64     ;
65 },

```

Cette fonction est appelée au clic sur le bouton du dark/light theme grâce à un écouteur d'événement posé dessus dans une méthode bind() initiée au chargement du module global app.js.



Ci-dessous on change les couleurs grâce à deux fonctions, l'une qui identifie la couleur à mettre et la passe en argument à l'autre.

Séparer cette seconde fonction qui applique le changement de couleur est intéressant pour pouvoir l'utiliser au chargement de la page par exemple.

Pour améliorer l'expérience utilisateur, les informations du choix de thème dark (ou retour à un thème clair) et de couleur sont enregistrées à chaque changement dans le local storage du navigateur. Ainsi, quand l'utilisateur ré-ouvre le site, il voit directement la page avec le dernier thème et la dernière couleur qu'il avait sélectionné.

```
67  /**
68   * Function calling the function that changes color theme when a color is selected
69   * It gives it the id of the button that has been clicked on
70   * @param {*} event
71   */
72  handleSelectColor: function(event) {
73      theme.changeColor(event.currentTarget.id);
74  },
75
76  /**
77   * Function applying the color theme.
78   * @param {string} color must be written as theme-color and part of the colorList property
79   */
80  changeColor: function(color) {
81      // Removes potential existing class of color already applied to the body
82      for (const color of theme.colorList) {
83          theme.bodyElement.classList.remove(color);
84      }
85      // Adds the new color class to the body
86      theme.bodyElement.classList.add(color);
87      // Changes the source of the logo to also have the good-colored logo
88      theme.imgLogoElement.src = `img/logo-${color}.png`;
89      // Stores the choice of color in the local storage
90      storage.setLocal("colorTheme", color);
91  }
92  };
```

Le local storage est adapté ici car ce ne sont pas des données critiques en terme de sécurité, nous n'en avons pas besoin sur notre serveur et elles y resteront tant que l'utilisateur ne change pas de machine ou de navigateur mais il peut fermer et réouvrir son navigateur sans les perdre, ce qui est pratique.

Les méthodes setLocal et getLocal sont codées dans le module storage afin de ne pas avoir à se soucier des formats de data enregistrées vs les méthodes natives localStorage.getItem() :

```
6  /**
7   * Replaces localStorage.setItem(), handling other than string values
8   * @param {String} key
9   * @param {*} value
10  */
11  setLocal: function (key, value) {
12      localStorage.setItem(key, JSON.stringify(value));
13  },
14
15  /**
16   * Replaces localStorage.getItem(), handling other than string values
17   * @param {String} key
18   * @returns {*}
19   */
20  getLocal: function (key) {
21      return JSON.parse(localStorage.getItem(key));
22  },
```


Encart Newsletter :

```

44 handleDisplay: function(event){
45     event.preventDefault();
46     newsletter.asideElement.classList.toggle("newsletter--on");
47     newsletter.formElement.reset();
48     message.resetMsgFromDom(newsletter.asideElement);
49 },
50
51 handleClose: function() {
52     newsletter.asideElement.classList.remove("newsletter--on");
53 },

```

Une fonction `handleDisplay` posée en écouteur sur le bouton "Newsletter" dans la nav affiche et cache l'encart. Elle remet également à zéro le formulaire et un éventuel message d'erreur existant dans l'encart.

Une fonction `handleClose` posée en écouteur sur le bouton X de l'encart permet de le fermer.

À la soumission du formulaire, on empêche le rechargement de la page pour pouvoir réaliser les actions nécessaires et on vide d'éventuels messages de précédentes soumissions erronées.

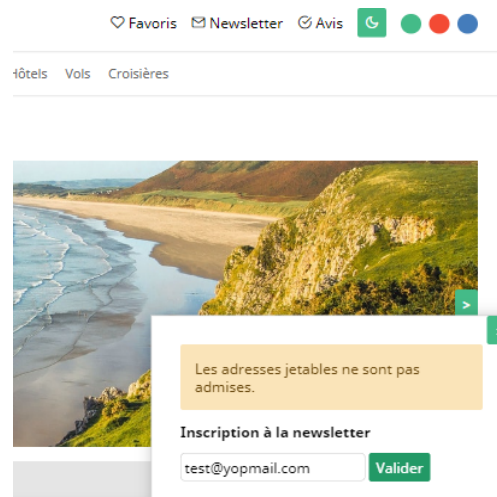
Il y a ici deux niveaux de vérification de l'input utilisateur gérés en front :

1. l'input du formulaire est de type="email" ce qui bloque la soumission d'une valeur vide ou d'un autre format que : xx@xx or xx@xx.xx.
 2. On vérifie que la valeur n'inclut pas une liste de domaines connus pour être des adresses emails jetables, spécifiées en propriété du module.
- Si c'est le cas, on affiche un message. Grâce à `toLowerCase` ce contrôle sera valide même si l'utilisateur entre le domaine en majuscule.

```

55 handleEmailSubmit: function(event) {
56     event.preventDefault();
57     message.resetMsgFromDom(newsletter.asideElement);
58
59     // Retrieves the value of the email filled by the user,
60     // removing blank spaces before and after and putting it to lower case
61     const email = newsletter.emailInputElement.value.trim().toLowerCase();
62     let isEmailForbidden = false;
63
64     // Checks the email for forbidden email domains
65     for (const forbiddenDomain of newsletter.forbiddenDomains) {
66         if (email.includes(forbiddenDomain)) {
67             message.addMsgToDom('Les adresses jetables ne sont pas admises.', newsletter.asideElement);
68             isEmailForbidden = true;
69         }
70     }
71     if (!isEmailForbidden) {
72         // Here we will put the instructions connecting to the back with an API call to add the email to the database
73         // (removing special characters before sending and checking if it already exists is done in the back)
74         // Also we will handle a success message to make the user know if it worked or not
75         console.log('Inscription de ' + email + ' réussie !');
76         event.currentTarget.reset();
77         newsletter.handleClose();
78     }
79 }
80 };

```



2. Précisez les moyens utilisés :

L'écriture de code s'est faite en JavaScript en s'appuyant sur les éléments HTML et CSS déjà écrits.

Pour débbuger, j'ai utilisé des `console.log()` et leur affichage dans la console des DevTool de Chrome, à des endroits stratégiques si j'avais une idée du problème ou de façon méthodique à chaque étape en cas de plus grosse difficulté.

3. Avec qui avez-vous travaillé ?

Sur cet exercice, nous avons commencé le travail en pair-programming tout comme pour l'exercice de la CP3 sur les fonctionnalités suivantes : thème dark, slider, newsletter.

Puis, à la fin de la session de pair-programming, j'ai continué à travailler seule dans le cadre des cours pour améliorer le code :

- passage d'un fichier script avec module unique à plusieurs modules ;
- amélioration de la logique dans les fonctionnalités ;

et seule au travers d'exercices supplémentaires après les cours pour les ajouts suivants :

- rajout de thèmes de couleur avec gestion du logo et stockage des thèmes dans le `localStorage` ;
- Tri de commentaires selon le nombre d'étoiles attribuées.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *École O'Clock*

Chantier, atelier, service ▶ *Exercice de formation "atelier TripAdvisor"*

Période d'exercice ▶ Du : *26/02/2024* au : *03/03/2024*

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée.

CP 5 - Mettre en place une base de données relationnelle

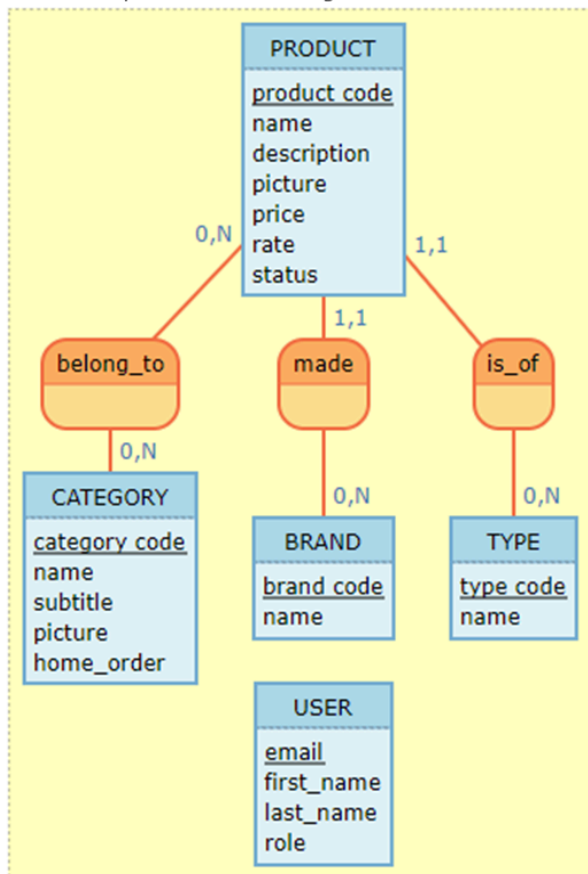
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors d'une série d'exercices simulant la réponse à une demande d'un client de créer une boutique en ligne de chaussure (qui servira pour les 3 prochaines CP, brief en annexe 5.1), le premier exercice consistait à modéliser et mettre en place la base de données qui allait servir au site.

Afin de bien comprendre les interactions des données du projet, éviter les redondances et l'hétérogénéité des datas, et simplement gagner en efficacité sur le long terme, la modélisation a été faite par étape :

1. MCD (Modèle Conceptuel de Données) :

Schéma représentant ce MCD grâce à l'outil MOCODO :



En analysant le brief client, on identifie des entités pertinentes, c'est-à-dire des choses concrètes ou abstraites qui existent de façon indépendantes, distinguables les unes des autres. Ici, il s'agit des produits, des catégories, des types, des marques et des utilisateurs (en majuscule sur le schéma). Ces entités comprennent des attributs, c'est-à-dire des données élémentaires (non dérivées d'un autre attribut) et atomiques (chaque occurrence d'entité n'a qu'une seule valeur de chaque attribut). Chaque entité doit avoir un attribut qui identifie les occurrences de façon unique (souligné sur le schéma). On identifie ensuite les relations entre les entités par une association (un verbe) et des cardinalités qui identifient le minimum (0 ou 1) et maximum (1 ou N) de relation par association d'entité. Par exemple :

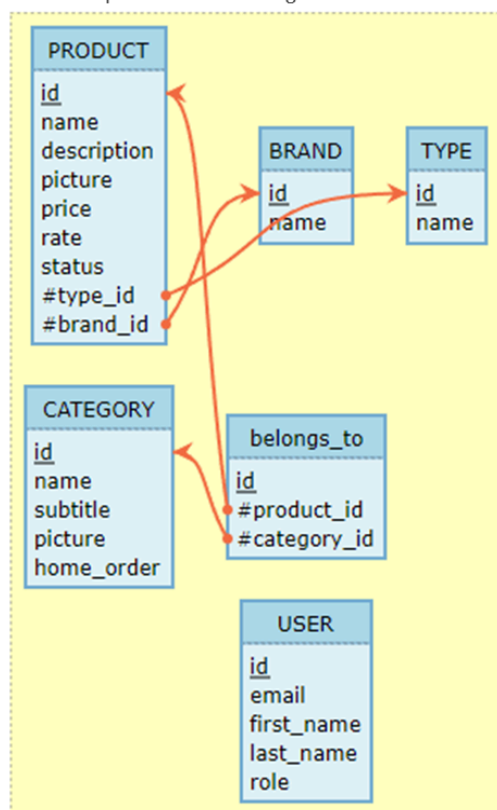
- Un produit ne peut avoir qu'une seule marque et en a forcément une (1,1) mais une marque peut être attribuée à plusieurs produits et n'avoir pas encore de

produit dans la BDD (0,N)→ Il s'agit d'une relation One to Many ;

- Un produit peut avoir zéro ou plusieurs catégories (0,N) et une catégorie peut être attribuée à zéro ou plusieurs produits (0,N) → Il s'agit d'une relation Many to Many.

2. MLD (Modèle Logique de Données) :

Schéma représentant ce MLD grâce l'outil MOCODO :



Ce modèle introduit des règles de logique de base de données pour rapprocher notre modèle de la BDD concrète :

- Au niveau du vocabulaire, les entités deviennent des tables et les attributs des colonnes ;
- On intègre une clé primaire pour chaque table et l'attribut discriminant devient une colonne comme une autre (soulignées) ;
- On intègre des clés étrangères aux tables ayant une cardinalité max de 1 vers une autre table (avec #);
- On intègre des tables de liaison entre les tables ayant une relation Many to many (belongs_to).

- PRODUCT (id, name, description, picture, price, rate, status, #brand_id, #type_id)
- belongs_to (id, #product_id, #category_id)
- CATEGORY (id, name, subtitle, picture, home_order)
- BRAND (id, name)
- TYPE (id, name)
- USER(id, email, first_name, last_name, role)

3. Dictionnaire de données :

Avec des tableaux dans un markdown, on liste les colonnes (champs) dans chaque table pour spécifier les informations techniques nécessaires : type, clé primaire, auto-incrémentation, unicité, nullité, valeur par défaut, description, commentaire.

On ajoute également les champs created_at et modified_at qui permettent une meilleure gestion de la BDD dans le temps. Ci-dessous l'exemple pour la table product :

Produits (product)

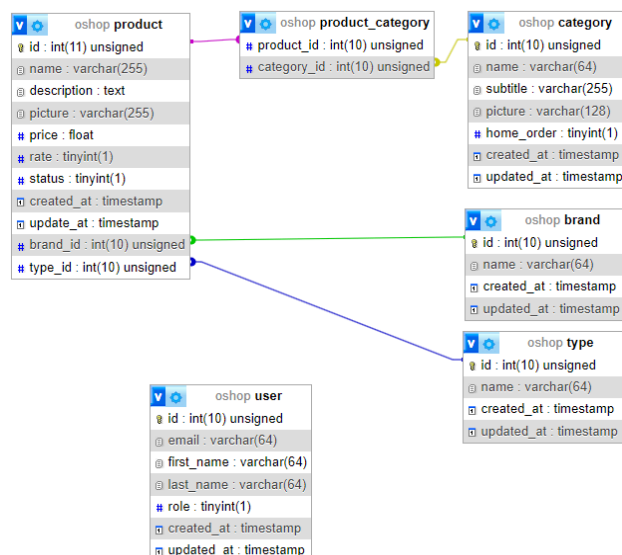
Champ	Type	Spécificités	Description
id	INT	PRIMARY KEY, NOT NULL, UNSIGNED, AUTO_INCREMENT	Product's id
name	VARCHAR(255)	NOT NULL	Product's name
description	TEXT	NULL	Product's description
picture	VARCHAR(255)	NULL	Product's picture URL
price	FLOAT	NOT NULL, DEFAULT 0	Product's price
rate	TINYINT(1)	NOT NULL, DEFAULT 0	Product's rating, from 1 to 5
status	TINYINT(1)	NOT NULL, DEFAULT 0	Product's status (1 = available, 2 = not available)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Product's creation date
updated_at	TIMESTAMP	NULL	Last product's update date
brand_id	INT	FK(brand), NOT NULL	Product's brand
type_id	INT	FK(type), NOT NULL	Product's type

DOSSIER PROFESSIONNEL (DP)

4. Création de la BDD et MPD (Modèle Physique de Données) :

J'ai créé la base de données grâce à l'outil phpMyAdmin (on pourrait également écrire directement un script de création SQL, il est d'ailleurs récupérable par phpMyAdmin si besoin de recréer une base similaire, script en annexe 5.2).

On peut récupérer sur phpMyAdmin la représentation du modèle physique de données :



2. Précisez les moyens utilisés :

J'ai utilisé Mocodo pour les représentations du MCD et MLD, markdown pour le dictionnaire de données et j'ai créé la base de données grâce à phpMyAdmin.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule, dans le cadre d'un exercice de formation le soir après les cours.

4. Contexte

Nom de l'entreprise, organisme ou association - École O'Clock

Chantier, atelier, service - Exercice de formation "oShop"

Période d'exercice - Du : 26/03/2024 au : 03/04/2024

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée.

CP 6 - Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Toujours dans le cadre de la série d'exercices simulant la réponse à une demande d'un client de créer une boutique en ligne de chaussures, j'ai développé un back-office pour la gestion des données. Le back-end étant en php, la connexion à la BDD se fait avec l'objet PDO et j'ai suivi le pattern Active Record qui représente les datas qu'on va manipuler sous forme d'objet.

<pre>◆ .gitignore 1 # Composer 2 vendor 3 4 # Fichier de config 5 app/config.ini</pre>	<pre>1 DB_HOST=localhost 2 DB_NAME=oshop 3 DB_USERNAME=oshop 4 DB_PASSWORD=oshop</pre>	Afin de protéger les identifiants de connexion à la BDD, on les place dans un fichier config.ini intégré dans le .gitignore.
--	--	--

Pour accéder aux données de la BDD, j'ai créé des méthodes pour chacune des actions du CRUD, ici en exemple pour les catégories :

- C -> Create : pour ajouter une instance de l'objet Category à la DB ;
- R -> Read : pour lire, c'est-à-dire pouvoir récupérer une ou plusieurs entrées de la table Category afin de pouvoir la ou les afficher par la suite ;
- U -> Update : pour modifier une entrée de la table Category (annexe 6.1) ;
- D -> Delete : pour supprimer une entrée de la table Category (annexe 6.2).

CREATE :

```
97 public function insert()
98 {
99     $pdo = Database::getPDO();
100     $sql = "
101         INSERT INTO category (`name`, subtitle, picture)
102         VALUES (:name, :subtitle, :picture)
103     ";
104
105     $stmt = $pdo->prepare($sql);
106     $stmt->bindParam(':name', $this->name);
107     $stmt->bindParam(':subtitle', $this->subtitle);
108     $stmt->bindParam(':picture', $this->picture);
109
110     if ($stmt->execute()) {
111         $this->id = $pdo->lastInsertId();
112         return true;
113     }
114     return false;
115 }
```

Ci-contre la méthode dans le model Category qui contient la requête SQL d'ajout à la DB.

Pour sécuriser l'ajout de données en se protégeant des injections SQL, on prépare les requêtes avec prepare(), bindParam() et on utilise execute().

DOSSIER PROFESSIONNEL (DP)

```

28 public function create()
29 {
30     $name = trim(filter_input(INPUT_POST, 'name', FILTER_SANITIZE_SPECIAL_CHARS));
31     $subtitle = trim(filter_input(INPUT_POST, 'subtitle', FILTER_SANITIZE_SPECIAL_CHARS));
32     $picture = trim(filter_input(INPUT_POST, 'picture', FILTER_SANITIZE_SPECIAL_CHARS));
33
34     $errors = [];
35
36     if(is_null($name)) {
37         $errors[] = "Le champs nom est manquant.";
38     }
39     if(is_null($subtitle)) {
40         $errors[] = "Le champs sous-titre est manquant.";
41     }
42     if(is_null($picture)) {
43         $errors[] = "Le champs image est manquant.";
44     }
45     if(mb_strlen($name) < 3 || mb_strlen($name) > 64) {
46         $errors[] = "Le nom doit contenir entre 3 et 64 caractères.";
47     }
48     if (!empty($subtitle)) {
49         if(mb_strlen($subtitle) < 5 || mb_strlen($subtitle) > 64) {
50             $errors[] = "Le sous-titre doit contenir entre 5 et 64 caractères.";
51         }
52     }
53     if (!empty($picture)) {
54         if(!str_starts_with($picture, 'http://') && !str_starts_with($picture, 'https://')) {
55             $errors[] = "L'url de l'image doit commencer par http:// ou https://.";
56         }
57     }
58
59     $category = new Category();
60     $category->setName($name);
61     $category->setSubtitle($subtitle);
62     $category->setPicture($picture);
63
64     if(empty($errors)) {
65         if($category->insert()) {
66             header("Location: " . $this->router->generate('category-list'));
67             exit;
68         }
69         $errors[] = "Erreur lors de l'ajout de la catégorie.";
70     }
71
72     $this->show('category/form', [
73         'errors' => $errors,
74         'category' => $category
75     ]);
76 }

```

Ci-contre, la méthode dans le contrôleur CategoryController qui récupère les informations du formulaire d'ajout, instancie un objet \$category, lui attribue les valeurs du formulaire et exécute la méthode insert() ci-dessus pour faire l'ajout à la BDD.

Pour se protéger des injections venant des données utilisateur, on les "nettoie" au moment de la récupération grâce à filter_input() avec FILTER_SANITIZE_SPECIAL_CHARS qui retire les caractères permettant les injections tels que ', ", <, > et & .

On contrôle également les champs au niveau de leur remplissage effectif, du nombre de caractères permis et du contenu tout en renvoyant un message d'erreur si les conditions ne sont pas respectées.

```

197 $router->map(
198     'POST',
199     '/category/add',
200     [
201         'method' => 'create',
202         'controller' => '\App\Controllers\CategoryController'
203     ],
204     'category-create'
205 );

```

Cette méthode create() du CategoryController est appelée dans le cas où on est dans une route POST où le chemin est /category/add. Ce cas est appelé quand on clique sur le bouton submit du formulaire d'ajout.

READ :

```
25 public static function find($id, $table, $class)
26 {
27     $pdo = Database::getPDO();
28
29     $sql = '
30         SELECT *
31         FROM ' . $table . '
32         WHERE id = ' . $id;
33
34     $pdoStatement = $pdo->query($sql);
35
36     $result = $pdoStatement->fetchObject('App\Models\\' . $class);
37
38     return $result;
39 }
40
41 public static function findAll($table, $class)
42 {
43     $pdo = Database::getPDO();
44     $sql = 'SELECT * FROM ' . $table;
45     $pdoStatement = $pdo->query($sql);
46     $results = $pdoStatement->fetchAll(PDO::FETCH_CLASS, 'App\Models\\' . $class);
47
48     return $results;
49 }
50
51 public static function findAllHomepage()
52 {
53     $pdo = Database::getPDO();
54     $sql = 'SELECT * FROM `category` ORDER BY id LIMIT 3';
55     $pdoStatement = $pdo->query($sql);
56     $results = $pdoStatement->fetchAll(PDO::FETCH_CLASS, 'App\Models\\Category');
57
58     return $results;
59 }
```

Les méthodes find et findAll sont mutualisées pour toutes les tables dans le CoreModel.

Ici, pas besoin de préparer la requête car il n'y a pas d'information venant de l'utilisateur.

Pour une requête SELECT, on utilise la méthode query() de PDO et on transforme son résultat en objet (car on est en POO) grâce aux méthodes fetch() et fetchAll() de PDOStatement.

Une méthode spécifique nous permet de ne récupérer que les 3 premières catégories pour avoir un affichage plus petit sur la page d'accueil.

2. Précisez les moyens utilisés :

J'ai utilisé les objets PDO et PDOStatement de PHP, le langage SQL et suivi le pattern Active record selon les principes de POO. Afin de déboguer, j'ai utilisé la fonction dump() de PHP et vérifié parfois l'évolution de la base de données grâce à l'interface phpMyAdmin.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule, dans le cadre d'un exercice de formation le soir après les cours.

4. Contexte

Nom de l'entreprise, organisme ou association - *École O'Clock*

Chantier, atelier, service - *Exercice de formation "oShop"*

Période d'exercice - Du : *26/03/2024* au : *03/04/2024*

5. Informations complémentaires (facultatif)

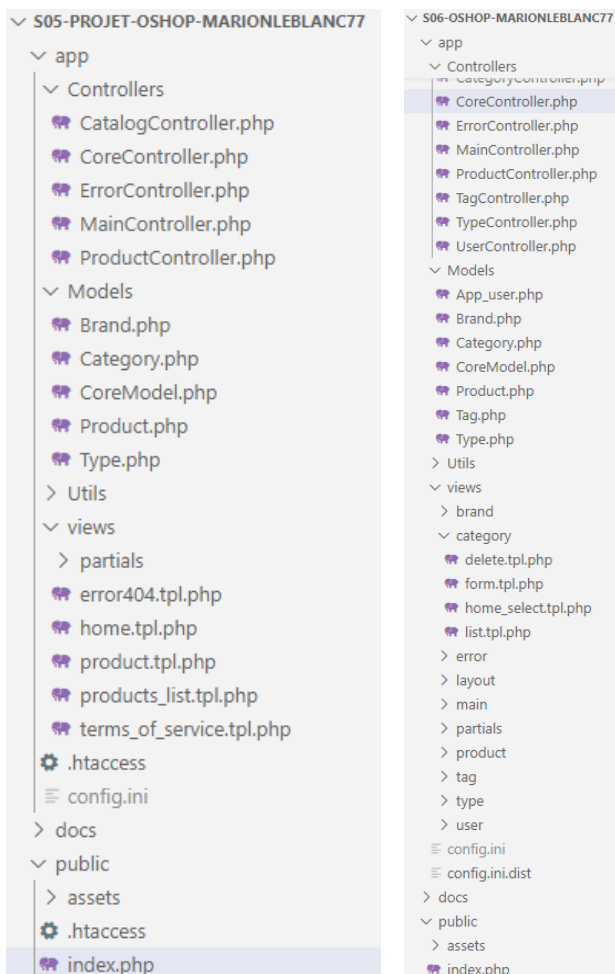
Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée.

CP 7 - Développer des composants métier côté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Toujours dans le cadre du même exercice, j'ai travaillé dans deux dossiers séparés, le site visible d'un côté (S05-PROJET-OSHOP) et le back-office de l'autre (S06-OSHOP). Ces deux dossiers suivent tous les deux le design pattern MVC, simple mais très lisible et organisé, avec une programmation orientée objet. J'ai donc découpé le code en 3 couches pour chaque dossiers + un Front Controller :



- Dossier Controllers : gère l'algorithme et la logique du site en utilisant une classe par entité du site (+ une classe CoreController pour la logique de fond, commune au site). Elles vont appeler des méthodes de Models en fonction de la logique du site, et quand le Model leur renvoie la data, elles appellent une View en lui fournissant cette data.

- Dossier Models : gère la data (avec le pattern Active record vu dans la CP6) avec une classe par table de la BDD (+ une classe mère avec les propriétés et méthodes communes à toutes les autres classes). Les objets instanciés par ces classes représentent les enregistrements des tables respectives. Le CRUD se fait par des méthodes de ces classes.

- Dossier Views : gère l'affichage. On mutualise les templates de code html autant que possible en utilisant des variables php qui vont contenir les données à afficher selon le chemin par lequel on a accédé au template.

- Le Front Controller est le fichier index.php dans le dossier public. C'est le point d'entrée unique vers l'application. Il fait le routage des requêtes et le dispatch aux contrôleurs.

Pour la suite de cette CP, je vais montrer des extraits de code du dossier du back-office.

Le Front Controller utilise les dépendances AltoRouter et AltoDispatcher :

```
22 // On instancie un objet $router de la class AltoRouter
23 // Cet objet va gérer les routes pour nous grâce à ses méthodes
24 $router = new AltoRouter();
25
26 // On vérifie si un chemin de base existe
27 if (array_key_exists('BASE_URI', $_SERVER)) {
28     // Si oui, on définit le paramètre basePath d'AltoRouter avec ce chemin
29     $router->setBasePath($_SERVER['BASE_URI']);
30 } else { // sinon
31     // On donne une valeur par défaut à $_SERVER['BASE_URI']
32     // (car on va l'utiliser dans le CoreController pour les assets)
33     $_SERVER['BASE_URI'] = '/';
34 }
35
36 // On mappe toutes les "routes" à AltoRouter, pour qu'avec une méthode de requête et une URL courante,
37 // il puisse appeler la bonne méthode du bon controller
38
39 // Routes MAIN
40 $router->map(
41     'GET', '/', [
42         'method' => 'home',
43         'controller' => MainController::class
44     ], 'main-home');
45
46
47 // Routes USER
48 $router->map(
49     'GET', '/user/login', [
50         'method' => 'login',
51         'controller' => UserController::class
52     ], 'user-login');
53
54 $router->map(
55     'POST', '/user/login', [
56         'method' => 'loginPost',
57         'controller' => UserController::class
58     ], 'user-login-post');
59
60 ...
61
62 // On appelle la méthode match() d'Altorouter qui renvoie un objet contenant un array 'target' avec la méthode et le Controller
63 // correspondant à la requête et l'URL courante qu'on stocke dans $match
64 $match = $router->match();
65
66
67 // On instancie un objet $dispatcher de la classe AltoDispatcher
68 // Cet objet prend les infos de $match (méthode et controller correspondant à la requête et l'URL)
69 // et une méthode+Controller à exécuter si $match n'est pas correcte,
70 // c'est-à-dire si la méthode match() d'AltoRouter n'a pas trouvé de route correspondante dans notre mapping.
71 $dispatcher = new Dispatcher($match, '\App\Controllers\ErrorController::err404');
72 // On donne en argument du Controller $router et $match pour qu'ils soient disponibles dans la méthode appelée
73 $dispatcher->setControllersArguments($router, $match);
74 // On exécute la méthode du Controller
75 $dispatcher->dispatch();
```

Tous les controllers héritent de la méthode show() de leur classe-mère pour appeler les views :

```

154 /**
155  * Méthode permettant d'afficher du code HTML en se basant sur les views
156  * @param string $viewName Nom du fichier de vue qu'on souhaite affiché
157  * @param array $viewData Tableau des données à transmettre aux vues
158  * @return void
159  */
160 protected function show(string $viewName, $viewData = [])
161 {
162     $categories = Category::findAll('category', 'Category');
163     $brands = Brand::findAll('brand', 'Brand');
164     $types = Type::findAll('type', 'Type');
165     // Comme $viewData est déclarée comme paramètre de la méthode show(), les vues y ont accès
166     // On la remplit avec les valeurs dont on a besoin sur TOUTES les vues
167     $viewData['currentPage'] = $viewName;
168     $viewData['assetsBaseUri'] = $_SERVER['BASE_URI'] . 'assets/';
169     $viewData['baseUri'] = $_SERVER['BASE_URI'];
170
171     // Si l'utilisateur est connecté, on récupère ses informations pour les passer aux views
172     if (isset($_SESSION['userId'])) {
173         $viewData['user'] = $_SESSION['userObject'];
174     }
175     // Sinon on instancie un objet vide qu'on met dans cette variable
176     else {$viewData['user'] = new App_user();}
177
178     // On veut désormais accéder aux données de $viewData, mais sans accéder au tableau
179     // On utilise la fonction extract() sur l'array $viewData pour pouvoir accéder à ses données sans passer par le tableau
180     // extract() crée une variable nommée $key avec la valeur de $viewData['key'] pour chaque élément du tableau $viewData
181     // Toutes ces variables $key seront disponibles dans les views
182     extract($viewData);
183
184     // On exécute les fichiers de views
185     // (1 seule fois et entraînant une erreur fatale stoppant le script si le fichier n'existe pas)
186     require_once __DIR__ . '/../views/layout/header.tpl.php';
187     require_once __DIR__ . '/../views/' . $viewName . '.tpl.php';
188     require_once __DIR__ . '/../views/layout/footer.tpl.php';
189 }

```

Voici deux exemples d'appel de show() pour voir le template du formulaire d'ajout ou d'édition de catégorie.

Selon si l'on clique sur le bouton "ajouter" ou "éditer", la méthode generate() d'AltoRouter génère un URL et une requête selon notre mapping.

```

2 <div class="container my-4">
3 <a href="= $this-&gt;router-&gt;generate('category-add')?" class="btn btn-success float-end">Ajouter</a>
4 <h2>Liste des catégories</h2>
5 <table class="table table-hover mt-4">
6 <thead>
7 <tr>
8 <th scope="col">#</th>
9 <th scope="col">Nom</th>
10 <th scope="col">Sous-titre</th>
11 <th scope="col">Photo</th>
12 <th scope="col">Ordre affichage accueil</th>
13 <th scope="col"></th>
14 </tr>
15 </thead>
16 <tbody>
17 <?php
18 foreach ($categories as $category) : ?>
19 <tr>
20 <th scope="row">= $category-&gt;getId() ?&gt;&lt;/th&gt;
21 &lt;td&gt;<?= $category-&gt;getName() ?&gt;&lt;/td&gt;
22 &lt;td&gt;<?= $category-&gt;getSubtitle() ?&gt;&lt;/td&gt;
23 &lt;td&gt;&lt;img style="height:2em" src="<?= $category-&gt;getPicture() ?&gt;"&gt;&lt;/img&gt;&lt;/td&gt;
24 &lt;td&gt;<?= $category-&gt;getHomeOrder() != 0 ? $category-&gt;getHomeOrder() : '' ?&gt;&lt;/td&gt;
25 &lt;td class="text-end"&gt;
26 &lt;a href="<?= $this-&gt;router-&gt;generate('category-edit', [ "categoryId" =&gt; $category-&gt;getId()])?" class="btn btn-sm btn-warning">
27 <i class="fa fa-pencil-square-o" aria-hidden="true"></i>
28 </a>

```

Les méthodes add et edit du CategoryController exécutent toutes les deux show() mais avec un paramètre category rempli différemment.

```

19 public function add()
20 {
21     $category = new Category();
22     $this->show('category/form', [
23         'category' => $category,
24         "csrfToken" => $_SESSION['token']
25     ]);
26 }

87 public function edit($id)
88 {
89     $category = Category::find($id, 'category', 'Category');
90     $this->show('category/form', [
91         "category" => $category,
92         "csrfToken" => $_SESSION['token']
93     ]);
94 }

```

La view affichent des éléments différents selon ce qui se trouve dans \$category (Titre différent et pré-remplissage si \$category est rempli) :

```

1 <div class="container my-4">
2 <a href="<?=$this->router->generate('category-list') ?>" class="btn btn-success float-end">Retour</a>
3 <h2><?=(is_null($category->getId())) ? 'Ajouter une catégorie' : 'Modifier la catégorie' :> . $category->getName(); ?> </h2>
4
5 <form action="" method="POST" class="mt-5">
6 <?php require_once __DIR__ . '/../partials/form_errors.tpl.php';?>
7 <input type="hidden" name="token" value="<?=$csrfToken?>">
8 <div class="mb-3">
9 <label for="name" class="form-label">Nom</label>
10 <input type="text" name="name" required="required" class="form-control" id="category-name" placeholder="Nom de la catégorie" value="<?=$category->getName()?>" aria-describedby="nameHelpBlock">
11 <small id="nameHelpBlock" class="form-text text-muted">
12     Entre 3 et 64 caractères.
13 </small>
14 </div>
15 <div class="mb-3">
16 <label for="subtitle" class="form-label">Sous-titre</label>
17 <input type="text" name="subtitle" required="required" class="form-control" id="category-subtitle" placeholder="Sous-titre" value="<?=$category->getSubtitle()?>" aria-describedby="subtitleHelpBlock">
18 <small id="subtitleHelpBlock" class="form-text text-muted">
19     Sera affiché sur la page d'accueil comme bouton devant l'image - entre 5 et 64 caractères.
20 </small>
21 </div>
22 <div class="mb-3">
23 <label for="picture" class="form-label">Image</label>
24 <input type="text" name="picture" class="form-control" id="category-picture" placeholder="image jpg, gif, svg, png" value="<?=$category->getPicture()?>" aria-describedby="pictureHelpBlock">
25 <small id="pictureHelpBlock" class="form-text text-muted">
26     URL relative d'une image (jpg, gif, svg ou png) fournie sur <a href="https://benoclock.github.io/506-images/" target="_blank">cette page</a>
27 </small>
28 </div>
29 <div class="d-grid gap-2">
30 <button type="submit" class="btn btn-primary mt-5">Valider</button>
31 </div>
32 </form>
33 </div>

```

Pour sécuriser les formulaires contre la faille CSRF, qui consiste à profiter de la session en cours pour envoyer un formulaire identique au nôtre à notre route en POST depuis un autre site et le remplir avec des valeurs malsaines, on va utiliser un système de token csrf.

```

70 $csrfTokenToCheckInGet = [
71     'user-list',
72     'product-list',
73     'category-list',
74     'type-list',
75     'tag-list',
76     'user-add',
77     'product-add',
78     'category-add',
79     'type-add',
80     'brand-add',
81     'tag-add',
82     'user-edit',
83     'product-edit',
84     'category-edit',
85     'type-edit',
86     'brand-edit',
87     'tag-edit',
88     'category-select',
89 ];
90
91 if(in_array($routeName, $csrfTokenToCheckInGet)){
92     $_SESSION['token'] = bin2hex(random_bytes(32));
93 }

```

Dans le CoreController, si on est dans la route en GET (affichage du formulaire), on crée une variable \$csrfToken aléatoire.

Dans le form, on inclut un champs input caché nommé "token" et on y met cette variable :

```

5 <form action="" method="POST" class="mt-5">
6 <?php require_once __DIR__ . '/../partials/form_errors.tpl.php';?>
7 <input type="hidden" name="token" value="<?=$csrfToken?>">
8 <div class="mb-3">
9 <label for="name" class="form-label">Nom</label>
10 <input type="text" name="name" required="required" class="fo

```

DOSSIER PROFESSIONNEL (DP)

Lorsqu'on est dans la route en POST (envoi du formulaire), on compare le token récupéré avec celui qui doit avoir été généré si le POST vient bien d'une de nos pages en GET et si ce ne sont pas les mêmes, on renvoie une erreur 403 d'accès non autorisé.

```
92 $csrfTokenToCheckInPost = [  
93     'user-create',  
94     'category-create',  
95     'brand-create',  
96     'type-create',  
97     'product-create',  
98     'tag-create',  
99     'user-update',  
100    'category-update',  
101    'brand-update',  
102    'type-update',  
103    'product-update',  
104    'tag-update',  
105    'category-select-post',  
106    'category-delete'  
107 ];  
108  
109 if(in_array($routeName, $csrfTokenToCheckInGet)){  
110     $_SESSION['token'] = bin2hex(random_bytes(32));  
111 }  
112  
113 else if(in_array($routeName, $csrfTokenToCheckInPost)){  
114     if (isset($_POST['token'])) {  
115         $token = filter_input(INPUT_POST, 'token', FILTER_SANITIZE_SPECIAL_CHARS);  
116     }  
117     elseif (isset($_GET['token'])) {  
118         $token = filter_input(INPUT_GET, 'token', FILTER_SANITIZE_SPECIAL_CHARS);  
119     }  
120     $sessionToken = isset($_SESSION['token']) ? $_SESSION['token']: '';  
121     if ($token != $sessionToken || empty($token)) {  
122         header('HTTP/1.0 403 Forbidden');  
123         $this->show('error/err403', ['user' => $_SESSION['userObject']]);  
124         exit;  
125     }  
126     else {  
127         unset($_SESSION['token']);  
128     }  
129 }
```

J'ai également répondu à la demande du client d'avoir un back-office nécessitant une connexion à un compte utilisateur avec deux rôles ayant accès à des pages différentes du back-office.

```

14 public function loginPost()
15 {
16     $email = filter_input(INPUT_POST, 'email', FILTER_SANITIZE_EMAIL);
17     $password = filter_input(INPUT_POST, 'password', FILTER_DEFAULT);
18
19     $errors = [];
20
21     if(empty($email)) {
22         $errors[] = "Le champs email est manquant.";
23     }
24
25     if(empty($password)) {
26         $errors[] = "Le champs mot de passe est manquant.";
27     }
28
29     $user = App_user::findByEmail($email);
30
31     if(($user !== false) && (password_verify($_POST['password'], $user->getPassword()))) {
32         $_SESSION['userObject'] = $user;
33         $_SESSION['userId'] = $user->getId();
34         header("Location: " . $this->router->generate('main-home'));
35         exit;
36     }
37     else {
38         $errors[] = "L'utilisateur n'existe pas où le mot de passe est incorrect";
39     }
40     $this->show('user/login', [
41         'errors' => $errors,
42     ]);
43 }

```

Lors de la soumission du formulaire de connexion, après avoir nettoyé les champs d'éventuels caractères spéciaux (faille XSS) et vérifié qu'ils étaient bien remplis, on vérifie que l'email existe dans la BDD et que le mdp associé et bien celui renseigné (grâce à password_verify on compare des mdp hachés).

Puis si tout est bon, on stocke les informations de l'utilisateur dans la constante prédéfinie PHP \$_SESSION.

```

23 $acl = [
24     'main-home' => ['admin', 'catalog-manager'],
25     'user-list' => ['admin'],
26     'user-edit' => ['admin'],
27     'user-update' => ['admin'],
28     'user-add' => ['admin'],
29     'user-create' => ['admin'],
30     'user-delete' => ['admin'],
31     'category-list' => ['catalog-manager', 'admin'],
32     'category-add' => ['catalog-manager', 'admin'],
33     'category-create' => ['catalog-manager', 'admin'],
34     'category-edit' => ['catalog-manager', 'admin'],
35     'category-update' => ['catalog-manager', 'admin'],
36     'category-delete' => ['catalog-manager', 'admin'],
37     ...
38     'brand-update' => ['catalog-manager', 'admin'],
39     'brand-delete' => ['catalog-manager', 'admin'],
40 ];
41
42 if (array_key_exists($routeName, $acl)) {
43     $authorizedRoles = $acl[$routeName];
44
45     $this->checkAuthorization($authorizedRoles);
46 }

```

Dans le CoreController, on liste les routes en leur attribuant un tableau avec les rôles y ayant accès.

Puis, si le nom de la route passé au CoreController est bien dans cette liste \$acl, on récupère les rôles autorisés qu'on soumet en argument de la méthode checkAuthorization ci-dessous.

```

132 protected function checkAuthorization($roles=[]) {
133     if (isset($_SESSION['userId'])) {
134         $currentUser = $_SESSION['userObject'];
135         $currentUserRole = $currentUser->getRole();
136
137         if (in_array($currentUserRole, $roles)) {
138             return true;
139         }
140
141         else {
142             header('HTTP/1.0 403 Forbidden');
143             $this->show('error/err403', ['user' => $_SESSION['userObject']]);
144             exit;
145         }
146     }
147
148     else {
149         header('Location: ' . $this->router->generate('user-login'));
150         exit;
151     }
152 }

```

checkAuthorization() vérifie que le rôle de l'objet de l'utilisateur stocké en session est bien dans le tableau de rôles autorisés passés en argument.



DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Pour cet exercice j'ai utilisé le langage PHP avec les classes AltoRouter (disponible sur Github) et AltoDispatcher (class d'une librairie créée par un professeur d'O'Clock).

J'ai mis en place une architecture MVC avec les principes de la POO et Active record.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule, dans le cadre d'un exercice de formation le soir après les cours.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *École O'Clock*

Chantier, atelier, service ▶ *Exercice de formation "oShop"*

Période d'exercice ▶ Du : *26/03/2024* au : *03/04/2024*

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée.

CP 8 - Documenter le déploiement d'une application dynamique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mes révisions pour l'apprentissage du métier et pour le TP, j'ai voulu redéployer avec Docker un des exercices fait en cours (une to-do-list faite avec Laravel comprenant un dossier front et un dossier back) sur une VM "vierge" Ubuntu installée sur Oracle VM VirtualBox.

1. Installation de Docker sur mon ordinateur et sur la VM

J'ai commencé par installer Docker :

- sur mon ordinateur selon le guide d'installation <https://docs.docker.com/desktop/install/windows-install/> du Docker Desktop en téléchargeant le fichier et suivant les instructions ;
- sur ma VM Ubuntu en suivant le guide d'installation <https://docs.docker.com/engine/install/ubuntu/> depuis le terminal avec les commandes suivantes :

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
# Add the repository to Apt sources:
echo \
"deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

On vérifie que l'installation s'est bien passé, incluant docker compose en vérifiant les versions :

```
docker --version
docker compose version
```

2. Création de l'image pour le back avec un Dockerfile et utilisation de Docker Compose comme script de compilation des images et de lancement des containers

Dans un fichier Dockerfile à la racine du dossier de la partie back-end de l'application, j'ai écrit les instructions pour Docker afin qu'il crée une image personnalisée à partir de l'image existante PHP avec Apache qui permettra de faire fonctionner le back-end :

```

1  # On part de l'image existante dans les images Docker : PHP 8.2 avec Apache
2  FROM php:8.2-apache
3
4  # On active l'extension PHP pdo_mysql
5  RUN docker-php-ext-install pdo_mysql
6
7  # On installe l'utilitaire pour dézipper & l'extension PHP zip
8  RUN apt update
9  RUN apt install -y libzip-dev zip
10 RUN docker-php-ext-install mysqli zip
11
12 # On se place dans le dossier /var/www/html
13 WORKDIR /var/www/html
14 # Et on copie le contenu du dossier courant (dossier backend) à l'intérieur de l'hôte
15 # dans le dossier courant à l'intérieur de l'image (dossier /var/www/html)
16 COPY . .
17
18 # On récupère Composer depuis une image Docker officielle
19 # et on le copie dans notre image (à l'emplacement /usr/bin/composer)
20 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
21
22 # On copie notre .env.example en un .env dans notre image
23 RUN cp .env.example .env
24 # On lance la commande `composer install` pour installer les dépendances
25 RUN composer install --no-interaction --optimize-autoloader
26 # On lance la commande `php artisan key:generate`, nécessaire au bon fonctionnement de Laravel
27 RUN php artisan key:generate
28
29 # On ajoute les droits d'écriture à tout le monde sur le dossier /var/www/html/storage,
30 # pour que Laravel puisse écrire ses logs et son cache
31 RUN chmod -R a+w /var/www/html/storage
32
33 # On modifie le DocumentRoot du virtual host par défaut d'apache
34 ENV APACHE_DOCUMENT_ROOT /var/www/html/public
35 RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf
36 RUN sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf
37
38 # On permet à Apache de lire les fichiers .htaccess et on active la réécriture d'URL
39 RUN sed -i 's/<Directory \/var\/www\/>/<Directory> s/AllowOverride None/AllowOverride all/' /etc/apache2/apache2.conf
40 RUN a2enmod rewrite

```

Puis, pour l'application au global, on va utiliser Docker Compose grâce à un fichier de configuration compose.yaml mis à la racine du dossier comprenant le front et le back. Ce fichier va indiquer à Docker Compose les services de notre application, c'est-à-dire les différents composants de l'application qui auront chacun leur image et leur conteneur (le back, le front et la base de données) :

```

1  services:
2      # service du front
3      frontend:
4          # On part de l'image existante Apache
5          image: httpd
6          # Instruction pour redémarrer automatiquement le conteneur s'il a un bug et se ferme
7          # (utile pour une vraie mise en production)
8          restart: always
9          # On redirige le port 8000 de l'ordinateur hôte vers le port 80 du conteneur
10         ports:
11             - "8000:80"
12         # On bind mount les fichiers du front
13         volumes:
14             - type: bind
15               source: ./frontend
16               target: /usr/local/apache2/htdocs
17     backend:
18         # On lance le build de notre image personnalisée pour le back,
19         # venant du Dockerfile
20         build: ./backend
21         restart: always
22         ports:
23             - "8080:80"
24         # On définit le fichier d'environnement
25         env_file:
26             - .env
27     db:
28         image: mariadb
29         restart: always
30         env_file:
31             - .env
32         # S'ils n'existent pas, on crée et stock sur l'hôte les volumes :
33         volumes:
34             # Le fichier de création de la DB
35             - ./DB.sql:/docker-entrypoint-initdb.d/DB.sql
36             # La Database (avec ses données)
37             - todolist-db:/var/lib/mysql:Z
38     volumes:
39         todolist-db:

```

3. Démarrage de l'application multi-conteneurs sur la VM Ubuntu (déploiement)

Pour lancer l'application, il suffira de quelques étapes :

- récupérer le projet de Github sur la VM Ubuntu (un simple téléchargement de fichier et dézippage dans notre cas car la VM est "vierge");
- copier le fichier .env.example du dossier global en un fichier .env et le remplir avec les informations de connexion à la base de données :

```

cp .env.example .env
nano .env

```

Puis remplir avec :

```

DB_HOST=db
DB_DATABASE=todolist
DB_USERNAME=root
DB_PASSWORD=todolist

```

DOSSIER PROFESSIONNEL (DP)

```
MARIADB_ROOT_PASSWORD=todolist
MARIADB_DATABASE=todolist
```

- lancer la commande suivante depuis la racine du dossier :

```
docker compose up -d
```

La première fois, il faudra éventuellement créer le docker group s'il n'existe pas et ajouter l'utilisateur au groupe puis log in dans le nouveau groupe :

```
sudo groupadd docker
sudo usermod -aG docker marion (ou un autre nom utilisateur)
newgrp docker
```

On peut voir l'application sur l'adresse localhost:8000 (et la page d'accueil de Laravel qui a bien démarré sur localhost:8080).

2. Précisez les moyens utilisés :

J'ai utilisé Docker (Engine, Compose et Desktop) et la ligne de commande. Ainsi qu'une VM Ubuntu sur Oracle VM VirtualBox. J'ai suivi les instructions vues en cours et la documentation de Docker.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seule après la formation, dans le cadre de la préparation de ce dossier.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *École O'Clock*

Chantier, atelier, service ▶ *Combinaison procédure exercice Pomodor'O et repo de l'exercice Laravel-monrepo*

Période d'exercice ▶ Du : 09/08/2024 au : 12/08/2024

5. Informations complémentaires (facultatif)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Baccalauréat général Série Scientifique Mention TRÈS BIEN	Académie de Créteil	20 juillet 2014
Programme Grande École Grade de Master	ESCP Europe	04 octobre 2021
Formation Développeur Web	École O'clock	19 juillet 2024



DOSSIER PROFESSIONNEL (DP)

Déclaration sur l'honneur

Je soussignée Anthonia Cosson, déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à Soliers City le 20/11/2024

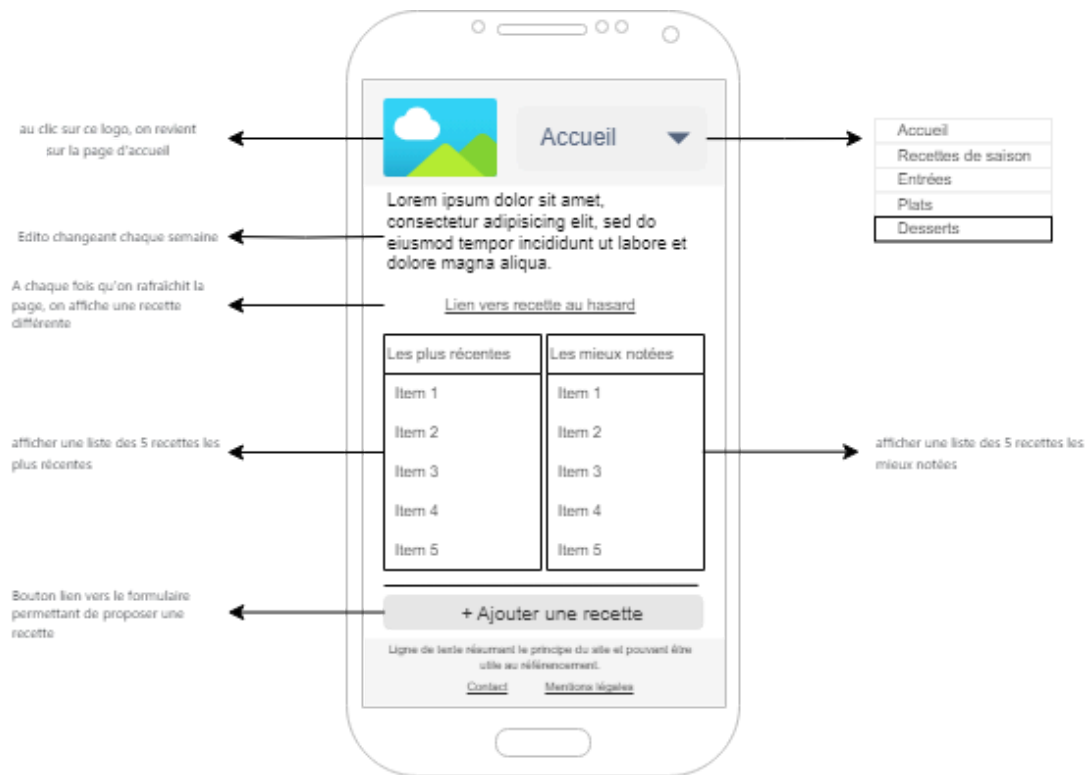
pour faire valoir ce que de droit.

Signature :

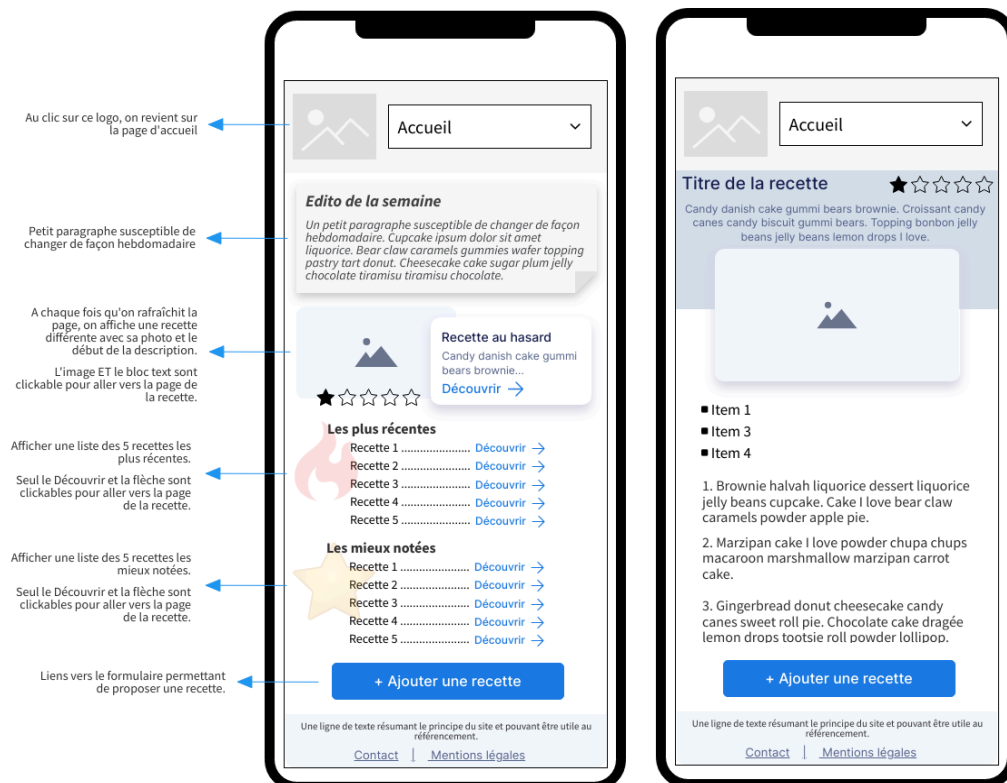
ANNEXES

(Si le RC le prévoit)

Annexe 2.1 : Wireframes avec draw.io



Annexe 2.2 : Wireframes avec mockflow (qui ne devrait pas avoir de couleurs et ce début de design)



Annexe 3.1 : Extrait du cahier des charges du projet de blog (partie pertinentes pour l'intégration du site statique)

Contexte

Présentation / Proposition initiale

Proposition de nom du projet : *oBlog*.

Au sein d'une école de développement web, un Responsable de formation souhaite donner accès à un *blog interne* aux élèves.

Objectif / Contextualisation

Le but du projet est de mettre à disposition un « blog de développeurs » pour que les étudiants qui le souhaitent puissent partager leur parcours de formation. Il s'agit donc d'un projet de blog tout ce qu'il y a de plus standard sur le plan technique, avec comme particularité que le but visé est surtout pédagogique (écriture et partage d'articles en interne dans l'école) :nerd_face:

[...]

Spécifications fonctionnelles

Apparence

- [...]
- Site responsive, *mobile-first*.

Contenus

- Des articles
- Des auteurs d'articles
- Des catégories d'articles (déjà connues : Front, Back, Collaboration, Vie du dev).

[...]

Interactions services

- Liens et/ou partages vers les réseaux sociaux.

Langue(s)

En français uniquement.

[...]

Navigation sur le blog

- Un menu principal (*responsive*) vers les catégories et l'accueil.
 - petits écrans : un menu « burger » dans lequel se trouvent les liens
 - sinon : liste de liens visibles
- On clique sur le titre ou l'extrait de texte de l'article pour en afficher le contenu complet.
- On peut cliquer sur une catégorie pour aller sur une page listant tous les articles de cette catégorie.
- Un menu secondaire (dans le footer ou ailleurs) pour lier vers Contact, Mentions légales & Admin.

Templates / charte graphique

Layout global

- Un menu avec :
 - Titre/logo
 - Liens vers les catégories
- Une image d'en-tête avec titre + slogan (baseline)
- "Sidebar" à droite qui regroupe :
 - Champ de recherche
 - Liste des catégories
 - Liste des auteurs
- Pied de page :
 - Liens vers les réseaux sociaux

- Liens vers admin, contact, mentions légales

Liste des articles

- La page d'accueil liste les derniers articles (nombre exact à déterminer).
- Pagination "Précédente" et "Suivante".
- Un article présente :
 - Un titre (cliquable)
 - Un résumé (cliquable)
 - Une date
 - Un auteur
 - Une catégorie

Contraintes techniques

- Site *responsive en mobile-first*.
- Compatibilité uniquement avec les dernières versions des navigateurs (Chrome, Firefox, Microsoft Internet Explorer 11 et Edge).

Spécifications techniques

Architecture logicielle choisie

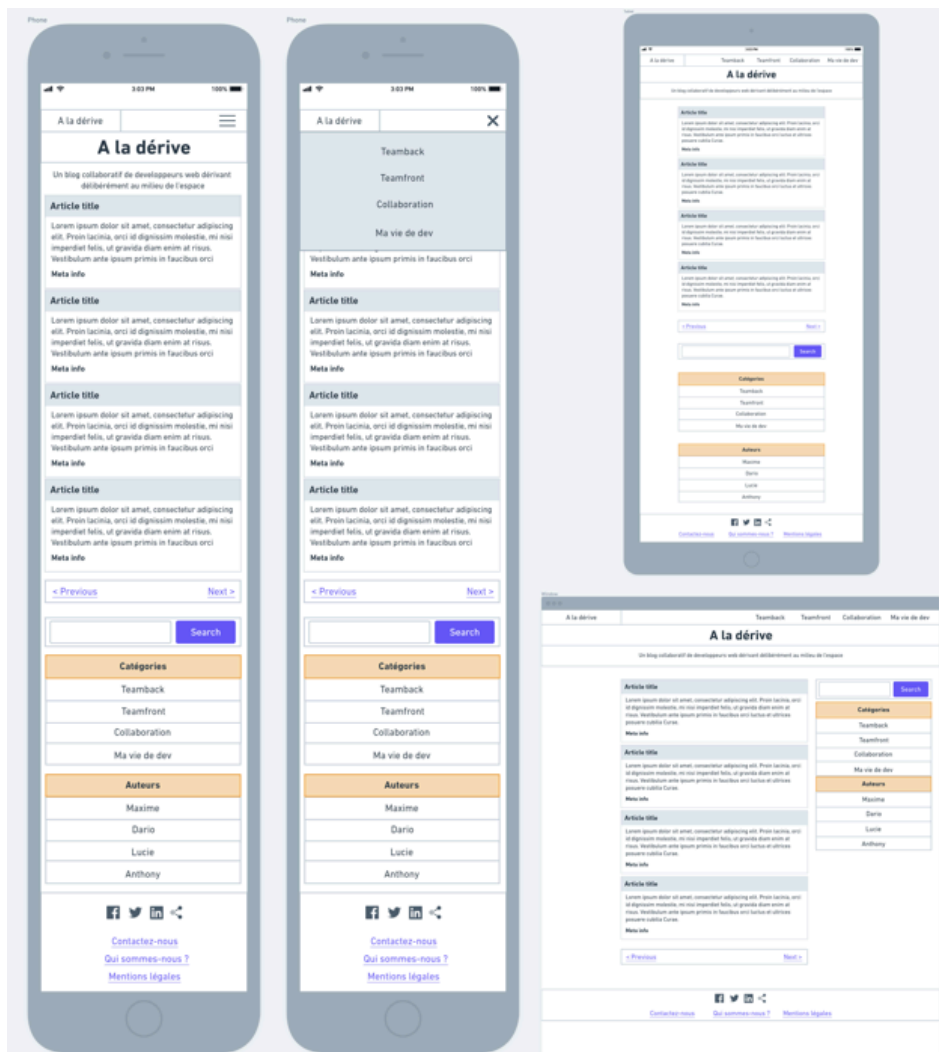
Le blog sera conçu avec :

Côté front

- HTML5 : le code respectera une sémantique correcte.
- CSS : dans sa version 2 ou 3, pour rester compatible avec Internet Explorer 11. L'utilisation d'un framework pour le *responsive/mobile-first* est envisagé.
- JavaScript : sera utilisé si besoin, avec parcimonie néanmoins (pas de fonctionnalités bloquantes).

[...]

Annexe 3.2 : Wireframes du projet blog



Annexe 4.1 : Code du module de Slider

```
1  export const slider = {
2    imagesList: [
3      "ocean.jpg",
4      "ski.jpg",
5      "city.jpg",
6      "road.jpg",
7      "nature.jpg",
8      "canyon.jpg",
9    ],
10   currentImagePosition: 0,
11
12   init: function(){
13     // When the page is loaded, calls the loadImages function to display the images in the slider
14     slider.loadImages();
15     slider.bind();
16   },
17
18   bind: function() {
19     // Puts event listener on the first and the second button
20     document.querySelectorAll('.slider_btn')[0].addEventListener('click', slider.handlePreviousSlide);
21     document.querySelectorAll('.slider_btn')[1].addEventListener('click', slider.handleNextSlide);
22   },
23
24   /**
25    * Function to create all the image elements of the slider with the images listed
26    * in the imagesList property and put the one defined in property
27    * currentImagePosition as first
28    */
29   loadImages: function() {
30     const sliderContainer = document.querySelector('.slider');
31     for (const image of slider.imagesList) {
32       const newImgElement = document.createElement('img');
33       newImgElement.src = 'img/' + image;
34       newImgElement.alt = 'Image ' + image;
35       newImgElement.classList.add('slider_img');
36       sliderContainer.append(newImgElement);
37     }
38     slider.goToSlide(slider.currentImagePosition);
39   },
40
41   /**
42    * Function to display the chosen image in the slider
43    * @param {number} newPosition
44    */
45   goToSlide: function (newPosition) {
46     const imageElements = document.querySelectorAll(".slider_img");
47     imageElements[slider.currentImagePosition].classList.remove("slider_img--current");
48     imageElements[newPosition].classList.add("slider_img--current");
49     slider.currentImagePosition = newPosition;
50   },
51
52   handlePreviousSlide: function () {
53     // If currentImagePosition is 0 (or less)
54     const newPosition = slider.currentImagePosition <= 0
55     // then newPosition takes the value of the lenght of the image list -1
56     // aka the last position to loop on the array of images
57     ? slider.imagesList.length - 1
58     // if not (the position is a positive number, there is room to go left),
59     // the new position just updates of -1 vs the current image
60     : slider.currentImagePosition - 1;
61     slider.goToSlide(newPosition);
62   },
63
64   handleNextSlide: function () {
65     // If currentImagePosition is higher than the lenght of the image list -1 (the last image)
66     const newPosition = slider.currentImagePosition >= slider.imagesList.length - 1
67     // then newPosition takes the 0 value to go back to the beginning
68     ? 0
69     // if not (there is room to go left), the new position just updates of +1 vs the current image
70     : slider.currentImagePosition + 1;
71     slider.goToSlide(newPosition);
72   },
73 };
```

Annexe 4.2 : Code du module des commentaires

```
1 export const reviews = {
2   init: function () {
3     reviews.bind();
4   },
5
6   bind: function () {
7     // Identifying the checkbox inputs that will be used as filters
8     const ratingInputs = document.querySelectorAll(".filters input[name=rating]");
9     // When they change the filtering action will be executed
10    for (const ratingInput of ratingInputs) {
11      ratingInput.addEventListener("change", reviews.handleRatingFilter);
12    }
13  },
14
15  handleRatingFilter: function (event) {
16    // Identifying the input that has been clicked with its value (given in html : 1, 2 or 3)
17    const rating = event.currentTarget.value;
18    // Selects the articles that have the identifying number (1, 2 or 3) as their data-rating attribute
19    const reviewArticles = document.querySelectorAll('.review[data-rating='${rating}']');
20
21    for (const reviewArticle of reviewArticles) {
22      // Toggles a class that hides the article on all the articles that match the clicked input
23      reviewArticle.classList.toggle("review--hidden");
24    }
25  },
26 };
```

Annexe 5.1 : Brief du client fictif

Sur toutes les pages

Navigation principale :

- Retour à l'accueil.
- Les catégories.
- Les types de produits.
- Les marques.

En bas de chaque page, il y aura :

- Le nom de la boutique.
- Le slogan.
- Les liens vers les réseaux sociaux.
- La mise en avant du fait que livraison et retours sont gratuits, que les clients ont 30 jours pour renvoyer leur produit, que les internautes peuvent contacter notre service client au 01 02 03 04 05 de 8h à 19h, du lundi au vendredi.
- Un formulaire d'inscription à la newsletter.

Catalogue

Voici le contenu du site prévu pour l'instant :

- Une page d'accueil (avec 5 catégories mises en avant).
- Une page des produits pour chaque catégorie (Détente, En ville, Au travail).
 - Une pagination sera présente.
 - La possibilité de filtrer par nom, note, ou prix.
- Une page produit.
- Une page des produits pour chaque type de produits (Chaussons, Escarpins, Talons aiguilles).
- Une page des produits pour chaque marque.

Panier d'achat

Sortie de ce brief pour des raisons de lisibilité.

DOSSIER PROFESSIONNEL (DP)

Commande

Sortie de ce brief pour des raisons de lisibilité.

Back-office

Zone réservée aux administrateurs *métier* et techniques du site.

- L'accès à cette zone nécessite de se connecter avec son compte
- Les échanges entre le navigateur et le serveur Web sont chiffrés par soucis de confidentialité & sécurité
- Gestion des catégories (liste, ajout, modification, suppression)
- Gestion des produits (liste, ajout, modification, suppression)
- Gestion des types de produits (liste, ajout, modification, suppression)
- Gestion des marques (liste, ajout, modification, suppression)
- Gestion des commandes (liste + changement du statut payé-envoyé-annulé-retourné)
- Gestion des 5 catégories en page d'accueil
- Gestion des utilisateurs du back-office
 - 2 types d'utilisateurs :
 - *catalog manager* pouvant gérer les données sur les produits du site (y compris catégories, types et marque)
 - *admin* pouvant, en plus de ce que peut faire un catalog manager, modifier le statut des commandes et créer des utilisateurs

Annexe 5.2 : Extrait de code de création de la BDD

```
21 -- Base de données : `oshop`
22 --
23 CREATE DATABASE IF NOT EXISTS `oshop` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
24 USE `oshop`;
25
26 -- -----
27
28 --
29 -- Structure de la table `brand`
30 --
31
32 DROP TABLE IF EXISTS `brand`;
33 CREATE TABLE `brand` (
34   `id` int(10) UNSIGNED NOT NULL COMMENT 'Brand's id',
35   `name` varchar(64) NOT NULL COMMENT 'Brand's name',
36   `created_at` timestamp NOT NULL DEFAULT current_timestamp() COMMENT 'Brand's creation date',
37   `updated_at` timestamp NULL DEFAULT NULL COMMENT 'Last brand's update date'
38 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
39
```

Annexe 6.1 : Code UPDATE du CRUD lié aux catégories

```
96 public function update($id)
97 {
98     $name = trim(filter_input(INPUT_POST, 'name', FILTER_SANITIZE_SPECIAL_CHARS));
99     $subtitle = trim(filter_input(INPUT_POST, 'subtitle', FILTER_SANITIZE_SPECIAL_CHARS));
100     $picture = trim(filter_input(INPUT_POST, 'picture', FILTER_SANITIZE_SPECIAL_CHARS));
101
102     $errors = [];
103
104     if(is_null($name)) {
105         $errors[] = "Le champs nom est manquant.";
106     }
107     if(is_null($subtitle)) {
108         $errors[] = "Le champs sous-titre est manquant.";
109     }
110     if(is_null($picture)) {
111         $errors[] = "Le champs image est manquant.";
112     }
113     if(mb_strlen($name) < 3 || mb_strlen($name) > 64) {
114         $errors[] = "Le nom doit contenir entre 3 et 64 caractères.";
115     }
116     if(!empty($subtitle)) {
117         if(mb_strlen($subtitle) < 5 || mb_strlen($subtitle) > 64) {
118             $errors[] = "Le sous-titre doit contenir entre 5 et 64 caractères.";
119         }
120     }
121     if(!empty($picture)) {
122         if(!str_starts_with($picture, 'http://') && !str_starts_with($picture, 'https://')) {
123             $errors[] = "L'url de l'image doit commencer par http:// ou https://.";
124         }
125     }
126
127     $category = Category::find($id, 'category', 'Category');
128     $category->setName($name);
129     $category->setSubtitle($subtitle);
130     $category->setPicture($picture);
131
132     if(empty($errors)) {
133         if($category->update()) {
134             header("Location: " . $this->router->generate('category-list'));
135             exit;
136         }
137         $errors[] = "Erreur lors de l'ajout";
138     }
139
140     $this->show('category/form', [
141         'errors' => $errors,
142         'category' => $category
143     ]);
144 }

124 public function update()
125 {
126     $pdo = Database::getPDO();
127     $sql = "
128         UPDATE category
129         SET `name` = :name,
130             subtitle = :subtitle,
131             picture = :picture,
132             updated_at = NOW()
133         WHERE id = :id
134     ";
135
136     $stmt = $pdo->prepare($sql);
137     $stmt->bindParam(':name', $this->name);
138     $stmt->bindParam(':subtitle', $this->subtitle);
139     $stmt->bindParam(':picture', $this->picture);
140     $stmt->bindParam(':id', $this->id);
141     return $stmt->execute();
142 }
```

Annexe 6.2 : Code DELETE du CRUD lié aux catégories

```
144 public function delete()
145 {
146     $pdo = Database::getPDO();
147
148     $sql = "
149         DELETE FROM `category`
150         WHERE id = :id
151     ";
152
153     $stmt = $pdo->prepare($sql);
154     $stmt->bindParam(':id', $this->id);
155
156     if ($stmt->execute()) {
157         $this->id = $pdo->lastInsertId();
158         return true;
159     }
160     return false;
161 }
```

```
146 public function delete($id)
147 {
148     $category = Category::find($id, 'category', 'Category');
149     if($category->delete($id)) {
150         header("Location: ". $this->router->generate('category-list'));
151         exit;
152     }
153     $errors[] ="Erreur lors de la suppression";
154
155     $this->show('category/list', [
156         'errors' => $errors,
157     ]);
158 }
```